

1) Industrial robot automate system.

i) Intermediate code design.

$t_1 = \text{Sensor} \langle \text{target} \rangle$
 $t_2 = \text{Obstacle} \cdot > \text{safe margin}$
 $t_3 = t_1 \text{ and } t_2$
RR t_3 goto L1
Goto L2

L1: motor speed = Normal-Speed

call move-ARM1

Goto L3

L2: motor speed = 0

call Emergency-Brake, 0

L3: ...

ii) modularity via procedure calls.

Procedure calls isolate discrete robotic tasks

iii) Role of compiler optimization: Eliminating redundant checks and loop invariant instructions guarantees low-latency sync.

2) Cybersecurity intrusion Detection system.

Efficient category handling with case statements instead of if-else chains (or) case

Statements compile into direct jump tables or binary search trees ($O(1)$ or $O(\log n)$). This allows compilers to branch directly.

Intermediate code design:

```
t1 = packet_port == UNTRUSTED_PORT
t2 = payload_size > MAX_LIMIT
t3 = t1 OR t2
IF t3 goto L1
L1: flag_threat = TRUE
    call log_alert, 1
    goto L3
L2: flag_threat = FALSE
    call PASS_PACKET, 1
L3: ...
```

3: Digital Payment Gateway System:

Intermediate code design:

```
t1 = User balance < transaction_amount
t2 = fraud_score < THRESHOLD
t3 = t1 AND t2
IF t3 goto L1
    goto L2
L1: Status = APPROVED
    call PROCESS_PAYMENT, 2
    goto L3
L2: Status = REJECTED
    call INITIATE_ROLLBACK, 1
L3: ...
```


ii) Backpatch in Conditional Processing:

During Boolean Exp translation, target label for success (true-list) or failure (false-list) are kept as list of unbound

iii) Scalability and security

A well designed compiler ensures accurate code translation and memory safety avoiding buffer overflows and double-spenders.

A) Smart Agriculture monitoring system.

i) Intermediate code design.

$t_1 = \text{Soil Moisture} < \text{MOISTURE-MIN}$

$t_2 = \text{temperature} > \text{temp-Max}$

$t_3 = t_1 \text{ AND } t_2$

If t_3 goto L1

goto L2

L1: pump status = ON

call ACTIVATE-IRRIGATION, 1

goto L3

L2: pump status = OFF

call DEACTIVATE-IRRIGATION, 1

L3: , ...

ii) Role of boolean expressions:

Boolean Expression combines multiple discrete sensor input into single logical evaluations.

iii) Optimization in Smart Agriculture:

Controllers often run on low power, resource constrained micro controllers.

5) Cloud based video Streaming platform.

i) Intermediate code design:

$t_1 = \text{current-bandwidth} < 2000$

PF t_1 goto L1

$t_2 = \text{current-bandwidth} < 5000$

PF t_2 goto L2

goto L3

L1: quality-level = LOW-RES

goto L4

L2: quality-level = MED-RES

goto L4

L3: quality-level = HIGH-RES

L4: call ADAPT-BITRATE, 1

ii) Case statements for quality levels

Case statement allow streaming engine to quickly select appropriate video bitrates based on changing bandwidth values.

iii) Significance in cloud multimedia systems

Optimizing intermediate code reduces server-side CPU utilization during real-time video transcoding and distribution.